

Local Multi-LLM Coding Workflow Evaluation

Mandatory Assignment 1 — Synopsis

Course	LLMs for Developers
Group	A
Members	Ahmad Alkaseb · Aleksandr Uktamovich Sorokin · Jeppe Jeppsson · Frederik Nicolajsen · Roman Ghazi · Kasper Amin
Date	06/10/2026

Objective: identify the most viable way to run a *local, multi-LLM* workflow that collaboratively produces software artifacts (architecture, implementation, testing, documentation, deployment validation) using local model backends, and recommend one approach with evidence.

1. Executive summary

Two open-source orchestration toolchains were built, run and compared on the same machine, against the same brief, using the same **two local Ollama endpoints**:

	Candidate A (recommended)	Candidate B
Toolchain	LangGraph — explicit state graph	CrewAI — role-based crew
Multi-endpoint	per-node <code>base_url</code> from config	per-agent LLM(<code>base_url=...</code>) from config
Control	full: plan/diff gate, bounded repair loop, git commit	limited: the framework drives the loop
Setup (measured)	≈10 min; 4 direct deps	≈25 min; ~120 transitive packages in a venv
Outcome	full artifact set, tests executed, deploy validated	artifacts only after extra glue; nothing executed

Recommendation: LangGraph (Candidate A) — it met every hard and functional requirement, and its explicit graph is what makes the required *predictability, reproducibility and context-management* guarantees achievable with small (7–8B) models.

2. Context and constraints

Host: 32 GB RAM, RTX 5070 Laptop (8 GB VRAM), Windows/PowerShell; Python 3.14 (Candidate A) and 3.12 in an isolated venv (Candidate B); Git and uv.

Two endpoints — `alpha = http://localhost:11434`, `beta = http://localhost:11435` (Ollama 0.35.1), both serving the same six role-tagged 7–8B Q4 models (`architect`, `techlead`, `coder`, `tester`, `docs`, `deploy`). This lets us bind different roles to different hosts and verify routing empirically.

Product brief (both candidates) — deliberately small so 7–8B models can produce coherent, runnable output: a **TaskFlow API**, a task/todo REST API in pure Python 3 stdlib (`http.server + json`) with `store.py` (`TaskStore`, CRUD), `service.py` (`TaskService`, validation), `api.py` (`make_server`) and `__main__.py`; a *Task* is a plain dict `{id, title, done}`.

The brief doubles as the **shared contract** — it pins every module signature, the primary defence against multi-agent interface drift (§4.4).

3. Candidate toolchains

3.1 Shortlist

LangGraph (code-first graph framework) and **CrewAI** (role-based crew framework) were shortlisted: they represent the two dominant orchestration paradigms — **explicit graph** vs **declarative crew**. Rejected: AutoGen/AG2 (heavy conversational loop, tool-calling-centric), n8n/Flowise (file-level codegen is awkward, weak diff review), OpenHands/Aider (not genuinely multi-role or multi-endpoint).

3.2 The two candidates

Candidate A — **LangGraph** models the workflow as a `StateGraph` (typed state, nodes, **conditional edges**). Nodes are ordinary Python, so file I/O, running tests and git are first-class; nodes bind independently to different models and endpoints; and conditional edges give an explicit, bounded **repair loop** driven by real test results.

Candidate B — **CrewAI** models it as a Crew of Agents (role/goal/backstory) and Tasks. Roles map naturally onto the six responsibilities and each agent can hold its own LLM (`base_url=...`), but the framework owns the execution loop, delegation and memory, so there is less control over *when* files are written and *how* failures are retried.

4. Evaluation

4.1 Setup complexity

	Candidate A (LangGraph)	Candidate B (CrewAI)
Install	<code>pip install pyyaml langgraph langchain-ollama pytest</code>	<code>uv venv --python 3.12 + uv pip install crewai</code>
Dependency count	4 direct	~120 transitive packages (litellm, tokenizers, textual, uvicorn...)

	Candidate A (LangGraph)	Candidate B (CrewAI)
Python-version risk	none (works on 3.14)	needs an isolated 3.12 venv — did not resolve cleanly on system Python 3.14
Time to first working run	≈ 10 minutes	≈ 25 minutes (mostly install/venv)

Moving parts — Candidate A is the two Ollama servers plus one script and one YAML file. Candidate B adds the CrewAI runtime stack (telemetry, embedding/memory machinery, a LiteLLM layer); even when unused these are installed, imported and add failure surface.

Verdict: Candidate A is materially simpler to stand up and to debug.

4.2 Capability coverage

Column C names the **native CrewAI feature** actually used, so “native vs glue” is explicit.

Responsibility	Candidate A (LangGraph)	Candidate B (CrewAI, native only)
Architecture & documentation	nodes → files on disk	Task(output_file=..., create_directory=True), one file per task
Tech lead tickets	tickets.json; ordering enforced by dependency batching	Task(output_pydantic=...) → structured JSON; ordering only <i>declared</i>
Implementation (N>2, multi-file)	dependency-ordered parallel batches, round-robin endpoints	async_execution=True + one Agent per parallel task; no dependency graph, no per-worker routing
Testing (create + run)	tester creates tests, run_tests executes pytest	creates test files; running them is not possible natively
Quality report	quality node	not possible natively
Deploy validation	deploy node executes the validator	artifacts only; execution not native
Control / diff review	--control review + diff.patch + git commit	not possible natively
Contract enforcement	prompt contract + executed tests	Task(guardrail=...) — native
Memory / context	scoped, dependency-ordered handoff	Crew(memory=True) — native, but see 4.2.1(8)

Responsibility	Candidate A (LangGraph)	Candidate B (CrewAI, native only)
Per-role endpoint binding	config lookup per call	LLM(base_url=...) per Agent — native

Correction to an earlier draft: CrewAI *can* run tasks concurrently via `async_execution=True`, so “parallelism requires extra Crews or delegation” was wrong. The real limits are **no dependency graph**, **no per-worker endpoint routing**, and **one file per task**.

4.2.1 What CrewAI cannot do by default (gaps, not worked around)

Absent from CrewAI 1.15.23 and therefore **not** implemented for Candidate B:

- 1. **Run tests** and capture structured results (our figures come from a host-side harness).
- 2. **Run the deployment validator** and record its exit code.
- 3. **git operations** — no native diff/commit, so no reviewable changeset.
- 4. **Plan + diffs review gate** before execution (the control requirement).
- 5. **A machine-generated quality report** (tests + static checks + limitations).
- 6. **Dependency-ordered execution** — `depends_on` can be written into output but nothing honours it.
- 7. **Deterministic per-worker endpoint routing** (*per-role* binding is native; no round-robin or failover).
- 8. **memory=True requires tool-capable models** — it adds *Search/Save memory* tools to **every** agent, which Ollama rejects (HTTP 400) for models without the `tools` capability (`tester:latest`); CrewAI then retries instead of falling back.

Verdict: Candidate A covers every responsibility *natively inside the workflow*. Candidate B covers the *generation* responsibilities natively but needs custom glue for anything that must **execute** (tests, deploy validation) or be **reviewed** (diffs, commits) — precisely the parts the brief marks as mandatory (*).

4.3 Multi-endpoint support

Both bind endpoints through the same config file, `config/endpoints.yaml`:

```
endpoints:
  alpha: { base_url: "http://localhost:11434" }
  beta:  { base_url: "http://localhost:11435" }
roles:
  # role -> (endpoint, model)
  architect: { endpoint: alpha, model: "architect:latest" }
  coder:     { endpoint: beta,  model: "coder:latest" }
  # ... techlead, tester, docs, deploy likewise
worker_pool: { endpoints: [alpha, beta], max_workers: 3 }
```

- **Candidate A** resolves (`base_url`, `model`) from config on **every call** and takes an `endpoint_override`; the parallel coding workers **round-robin** across [`alpha`, `beta`]. One config edit re-binds any role.
- **Candidate B** binds each Agent to LLM(`base_url=...`) at crew-build time.

Different roles on different endpoints: yes, in both. Config-driven switching is demonstrated by running Candidate A twice with `endpoints.yaml` vs `endpoints.alt.yaml` — identical except for the role/worker mappings:

Running the same workflow twice with the two profiles swapped **every** role (architect/techlead/docs moved 11434→11435

and coder/tester/deploy the inverse), the coding workers were observed on **both** endpoints in both runs, and both finished with DEPLOY OK (7 passed / 0 failed and 12 passed / 0 failed).

The per-call table in each `run_manifest.json` proves the switch: not one line of code changed between the runs.

4.4 Failure modes

Failures were **observed**, not hypothesised — each candidate was run and the first thing to break, plus the recovery path, was recorded.

Rank	Failure	Candidate A	Candidate B
1	Interface drift between workers	a worker invented <code>models.py</code> ; another called <code>store.create(dict)</code> where the contract said <code>create(title)</code>	same class of risk; sequential tasks reduce it but cost parallelism
2	Missing imports	<code>api.py</code> used <code>TaskStore()</code> unimported → <code>NameError</code>	same
3	Serialisation errors	<code>api.py</code> returned <code>str(dict)</code> instead of <code>json.dumps</code>	same
4	Wrong import root / structured-output drift	deploy script could not import <code>taskflow</code> ; tech lead sometimes wrapped JSON in prose	same

Detection and recovery. Candidate A's detection is **executable** — `run_tests` runs real `pytest`, `deploy` runs a real validator, `quality` runs `py_compile`; exit codes are the signal and nothing is taken on the model's word. Failures are recovered by a **conditional edge** into a bounded repair node that is *targeted* (only files named in the traceback), *informed* (given the traceback and its dependencies' code) and *bounded* (`max_repair_rounds: 3`); when the budget runs out the run **continues and reports** instead of looping. Candidate B's detection is **textual** (no exit code to branch on) and recovery means re-running the crew.

Observed progression (Candidate A, same brief and models):

Across iterations, the same brief and models went: **v1** (all tickets in parallel) 1 collection error, deploy FAILED → **v2** (explicit shared contract) 5 failed → **v3** (dependency-ordered batches + context handoff) 8 passed / 1 failed → **v4** (targeted repair + JSON contract) deploy OK → **v6** (JSON serialisation contract + JSON test bodies) **7 passed / 0 failed**, deploy OK.

Each failure was diagnosed from machine-readable evidence and fixed by changing the *workflow*, not by hand-editing generated code.

Candidate B, baseline run (`runs/runB_demo/`) — free-form crew, no native file features. Three gaps: (1) the **tech lead baked a wrong interface into its acceptance criteria** (`create_task, ...`), so coder and tester were self-consistent but contract-wrong (9 passed against the wrong interface); (2) docs stayed prose and, wrapped in one outer fence, only a **335-byte truncated README** materialised (no architecture, OpenAPI, ADR, api-usage or runbook); (3) nothing was executed or reviewed, and `api.py` was functionally broken (a bound method passed where a handler *class* is required; no `/healthz`; PUT instead of PATCH).

Candidate B, native run (`runs/runB_native_v3/`) — rebuilt with only native features. Materially better: **15/15 artifacts written**, and `output_pydantic` produced a **valid, contract-correct tickets.json** (fixing the baseline's core

defect). But two native limits appeared: `output_file` writes output **verbatim**, so every `.py` is wrapped in a markdown fence and **not directly runnable**; and `memory=True` **hung the full crew** (~18 min, no model activity — an isolated single-task memory crew works). After harness-side de-fencing (glue): **17 passed / 4 failed / 6 errors** — `service.py` lacked the `ValueError` validation (guardrails check text, not behaviour) and `test_api.py` was broken.

CrewAI therefore *generates* well and its `output_pydantic/guardrail` primitives are genuinely useful, but it does not **execute, review, de-fence, or guarantee** the contract.

4.5 Recommendation (summary)

Choose Candidate A (LangGraph) — it is the only candidate that satisfies *all* the mandatory requirements, including the ones that must **execute** (tests, deploy validation) or **review** (plan + diff, commits). CrewAI remains a reasonable choice if the deliverable is *prose* rather than *executed software*. Full rationale in §9.

5. Architecture of the recommended workflow (Candidate A)

5.1 Shape

```

config/endpoints.yaml    endpoints {alpha,beta};  roles {role -> endpoint, model}
    | resolved per call
    v
architect > techlead > coders(N>2) > tester > run_tests --pass--> docs > deploy > quality > apply
(alpha)      (alpha)      (alpha+beta) (beta)      (pytest)          (alpha) (beta)
(diff+commit)
                                     |
                                     +--fail--> repair > run_tests (bounded, max
3)

```

5.2 Roles, endpoints and artifacts

Model-backed nodes bind to endpoints as **architect/techlead/docs** → **alpha** and **coder/tester/deploy** → **beta**, with the parallel coding workers round-robinning across both. What each produces:

- **architect** → `architecture.md`, `adr/0001-*.md`, `api/openapi.yaml`; **techlead** → `tickets.json`
- **coders** (parallel) → `src/taskflow/{store,service,api,__main__}.py`; **tester** → `tests/test_{store,service,api}.py`
- **docs** → `README.md`, `docs/api-usage.md`, `docs/runbook.md`; **deploy** → `Dockerfile`, `deploy/validate_deploy.py` (executed), `docs/environment.md`
- **quality / apply** (no model) → `reports/quality_report.md`, `diff.patch`, synced project, git commit

5.3 Parallelism, control and context

- **Parallelism (N > 2)**: tickets carry `depends_on`; the orchestrator groups them into **dependency-ordered batches** (Kahn). Independent tickets run **in parallel** in a `ThreadPoolExecutor` (`max_workers: 3`), each handed the contract plus the **code of its completed dependencies**, and each bound to an endpoint by **stable round-robin** so both servers are used. With the default ticket graph: batch 1 = `store.py`; batch 2 = a **three-worker batch** (`service.py`, `api.py`, `__main__.py`).
- **Control**: nodes write into `runs/<id>/staged/`; apply diffs it against the live project. `--control review` **never touches** the project; `--control apply` syncs and commits.

- **Context:** no node receives the whole repository (architect: brief only; tech lead: brief + truncated architecture; coders: ticket + dependency code; tester: code under test). Raw output goes to disk rather than accumulating, so prompt size grows with a ticket's **fan-in**, not repository size.
- **Reproducibility:** identical config + brief → identical run *structure* even though the text differs; each run is a self-contained folder with inputs, outputs, diff and manifest.

6. Evidence — what the recommended workflow actually produced

Reference run runs/runA_v6/; the generated project is committed to demo_project/. All six responsibilities are covered by real files (see §5.2 for which node produces what), plus the run evidence:

runs/<id>/07_test_output.txt (executed tests), 10_deploy_validation.txt (DEPLOY OK), quality_report.md, diff.patch, run_manifest.json and context_manifest.json.

Outcome (runA_v6): pytest exit 0 — **7 passed / 0 failed / 0 errors**; **11** files py_compiled, **0** syntax failures; deploy/validate_deploy.py exit 0 → **DEPLOY OK**; **0** repair rounds. The applied project is independently runnable (python -m pytest demo_project → 7 passed).

Routing evidence: runs/<id>/run_manifest.json records every model call with its role, resolved endpoint, model tag and latency; in the reference run the three parallel coding workers appear on **both** endpoints.

Candidate B evidence (CrewAI) — three runs preserved:

Three CrewAI runs are preserved: runB_demo/ (baseline, free-form text — 9 files scraped, docs lost, tests passing against a **wrong** interface), runB_native_v2/ (native + memory=True — **stalled**, no output) and runB_native_v3/ (native, --no-memory — **15/15 artifacts**, then **17 passed / 4 failed / 6 errors** after harness de-fencing).

7. Requirement-by-requirement compliance

Type	Requirement	How the recommended workflow meets it
Hard	≥ 2 separate local endpoints	config/endpoints.yaml declares alpha (11434) and beta (11435); both are live and used in the same run
Hard	Switch/route by configuration	routing is a data lookup; --config config/endpoints.alt.yaml re-binds every role with no code change
Hard	Open source	Ollama, LangGraph, LangChain and pytest (all MIT)
Functional	Architecture: decomposition, interfaces, topology, ADRs	architect emits architecture.md (components, interfaces, deployment topology, constraints), ADR-0001 and openapi.yaml
Functional	Tech lead: scope, acceptance criteria, ordering	techlead emits tickets.json; the graph <i>enforces</i> depends_on via dependency-ordered batches

Type	Requirement	How the recommended workflow meets it
Functional	Implementation: $N > 2$ workers, multi-file	<code>ThreadPoolExecutor(max_workers: 3)</code> , one worker per ticket, four files across <code>src/taskflow/</code>
Functional	Testing & quality: create + run tests, quality report	<code>tester</code> writes unit + integration tests; <code>run_tests</code> executes <code>pytest</code> ; <code>quality</code> composes results, <code>py_compile</code> checks and limitations
Functional	Documentation: README, API usage, runbook, design docs	<code>docs</code> writes <code>README.md</code> , <code>api-usage.md</code> , <code>runbook.md</code> ; design docs come from architect
Functional	Deploy validation: script / container / env docs	<code>deploy</code> writes <code>validate_deploy.py</code> , <code>Dockerfile</code> , <code>environment.md</code> and runs the validator
Non-functional	Predictability & control	- <code>control review</code> produces a plan + diff without touching the project ; node order is fixed by the graph
Non-functional	Reproducibility	each run is a self-contained folder with inputs, outputs, diff and manifest; changes arrive as a reviewable diff and a git commit
Non-functional	Context management	scoped, dependency-ordered context per node (<code>context_manifest.json</code>); raw output to disk; prompt size grows with ticket fan-in, not repo size
Non-functional	Security baseline	both endpoints are loopback-only; no public exposure, no cloud dependency

8. Tradeoffs, risks and failure modes

8.1 Tradeoffs

Decision	Benefit	Cost
Explicit graph over declarative crew	deterministic control, executable detection, bounded recovery	more code; you own the plumbing
Small, dependency-free target	7–8B models can produce runnable output	does not stress large-repo context handling
Prompt-level structured output instead of tool-calling	works with models that lack tool support	needs a tolerant parser; malformed output must be detected

Decision	Benefit	Cost
Dependency-ordered batching	removes the dominant interface-drift failure	reduces intra-batch parallelism for deep ticket graphs

8.2 Risks

- **Model quality dominates.** With 7–8B models correctness is probabilistic, and tolerant parsing can mask malformed output; the workflow’s value is that it **detects** failure mechanically (real tests, `py_compile`) and **bounds** recovery.
- **Latency and host contention.** A run makes ~15–20 model calls (slowest single call observed ≈230 s), and two Ollama servers on one GPU serialise model loads; genuine multi-host deployment would remove that bottleneck.
- **The generated Docker file is not validated.** Docker is not a prerequisite (the deploy requirement is met by the executed `validate_deploy.py` plus `docs/environment.md`), so no image is built; in the reference output it would also need `PYTHONPATH=/app/src`. Disclosed rather than hand-patched, so the artifacts remain purely workflow-generated.

8.3 Failure modes and mitigations (observed)

Failure mode	Detection	Mitigation
Interface drift between workers (invented <code>models.py</code> ; <code>create(dict)</code> vs <code>create(title)</code>)	<code>pytest ImportError</code> / type error	explicit contract naming every module; dependency-ordered batches hand over the real dependency code
Missing or wrong imports (unimported <code>TaskStore</code> ; deploy script cannot <code>import taskflow</code>)	<code>NameError</code> / <code>ModuleNotFoundError</code>	targeted repair from the traceback; contract states the import lines and <code>src</code>
Serialises with <code>str()</code> instead of <code>json.dumps()</code>	integration test <code>JSONDecodeError</code>	contract mandates <code>json.dumps</code> ; JSON test bodies
Model writes a broken test (<code>urlopen(..., method=...)</code>)	failing integration test	repair regenerates the <i>test</i> file, pitfall spelled out
Model wraps JSON in prose	<code>extract_json</code> returns <code>None</code>	tolerant extraction + logged fallback ticket set
CrewAI: <code>output_file</code> writes output <i>verbatim</i> (fences included)	generated <code>.py</code> raise <code>SyntaxError</code>	none native — needs a de-fencing step
CrewAI: <code>memory=True</code> adds <i>Search/Save memory</i> tools to every agent	HTTP 400 on non-tool models; retry storm, or a hung crew	- - <code>no-memory</code> , or re-bind to a tool-capable model
CrewAI: <code>async_execution=True</code> with one shared Agent; guardrails check text, not behaviour	Executor is already running; missing validation slips	one Agent per parallel task; stricter markers (cannot verify behaviour)

Failure mode	Detection	Mitigation
	through	

What breaks first, in one line: *the interface between two agents* — not the model, not the transport. In a multi-agent coding workflow, **the contract is the product**.

9. Recommendation and rationale

Adopt Candidate A (LangGraph). It is the only candidate that, out of the box, satisfies *all* the mandatory requirements — including those that must **execute** (tests, deploy validation) or **review** (plan + diff, commits) — and its explicit graph makes routing, sequencing, context scoping and bounded recovery *inspectable and testable* rather than emergent. CrewAI suits deliverables that are documentation and planning text, but it leaves *execution* and *review* to custom glue and is harder to reproduce.

Starting from CrewAI instead? Keep the same `config/endpoints.yaml` and shared contract, then wrap the crew’s output in a host-side executor (write files, run `pytest`, run the validator, produce a diff) — effectively re-implementing the LangGraph nodes.

10. Review of another group’s work

Required by the brief (“review one of the other deliveries”). Completed after the peer-review session, assessing the group’s delivery against the same requirements.

Review checklist used

1. ≥ 2 local endpoints with config-driven routing — with evidence?
2. Which of the six responsibilities are native vs custom glue?
3. Control mechanism (plan + diffs / ask-before-run), and re-runnable output with a diff or commit?
4. Reproduce their setup guide: did it work first time, and where did it fail?

(Peer review to be filled in with the assigned group’s delivery and findings.)

Full reproduction steps (two endpoints, config, demo run, verification): docs/setup-guide.md.